

Module 1

Malware

- **Malware** (short for "malicious software") is any code or application specifically designed to damage, disrupt, or gain unauthorized access to a computer system, network, or device.
- Digital "illness" that interferes with how a computer should normally function.

Types of Malware

Type	How it Works	Key Characteristic
Virus	Attaches to a clean file and spreads when a human executes that file.	Needs human action to spread.
Worm	A standalone program that replicates itself across a network.	Spreads automatically without human help.
Trojan	Disguises itself as legitimate software (e.g., a "free game").	Relies on deception to get installed.
Ransomware	Encrypts your files and demands a "ransom" payment to unlock them.	High financial impact; often uses crypto.
Spyware	Secretly monitors user activity (keystrokes, passwords, browsing).	Focuses on data theft and surveillance.
Adware	Floods your browser or desktop with unwanted advertisements.	Type
Rootkit	Gives an attacker administrative (root) access while remaining hidden.	Extremely difficult to detect.
Keylogger	Records every key you press to steal login credentials.	A specific subtype of spyware.

Malware Analysis

- Malware Analysis to understand how a specific piece of malware functions. There are three primary levels:

Malware Analysis Type	Details	Looks for
Static Analysis	This involves examining the malware without actually running it.	File headers, strings, imports, hashes
Dynamic Analysis	The malware is executed in a controlled environment (called a Sandbox) to see what it actually does.	File changes, registry modifications, network traffic
Hybrid Analysis	It combines both methods, starting with static analysis to gather basic info, and then using dynamic analysis to confirm behaviors.	File headers, strings, imports, hashes, File changes, registry modifications, network traffic
Reverse Engineering	The most advanced level. Analysts use debuggers and disassemblers to "deconstruct" the malware line-by-line. This is used for complex, never-seen-before (Zero-Day) attacks to understand the logic behind the malware's algorithm	Disassembles or decompiles malware to Understands logic

Best Analysis Method

Malware Type	Best Analysis Method
Virus	Static + Dynamic
Worm	Network + Behavioral
Trojan	Dynamic + Reverse engineering
Ransomware	Memory + Static + Crypto analysis
Fileless malware	Memory forensics
Rootkits	Kernel & memory analysis
Botnets	Network traffic analysis

Setting up Malware Lab

Virtualization

Virtualization is the process of creating a software-based (or "virtual") version of a physical computer. It allows you to split a single physical computer's resources like its CPU, RAM, and storage into multiple independent "Virtual Machines" (VMs)

Hypervisor

A **hypervisor** sits between the physical hardware and the virtual machines and manages how CPU, memory, storage, and network resources are shared. Hypervisor (also known as a Virtual Machine Monitor or VMM).

The hypervisor sits between the physical hardware and the VMs.

1.Abstract: It tricks the guest operating system into thinking it has its own dedicated physical CPU and memory.

2.Isolate: It ensures that if one VM crashes or gets a virus, the others (and the host machine) remain unaffected.

3.Allocate: It dynamically hands out hardware resources to whichever VM needs them at that moment.

Types of hypervisors

There are two types of hypervisors

Type	Name	Where it sits	Best For...	Examples
Type 1	Bare Metal	Directly on the hardware.	Large servers & Data centers.	KVM, VMware ESXi, Microsoft Hyper-V
Type 2	Hosted	On top of a regular OS (like Windows).	Personal use & Testing.	VirtualBox, VMware Workstation

Types of VM Networking

Mode	Communication	Internet Access?	Best Used For...
Bridged	VM acts as a real device on your physical Wi-Fi/LAN.	Yes	Running a web server or a printer.
NAT	VM "hides" behind your host IP (like a router).	Yes	Standard web browsing or downloading updates.
Host-Only	Only the Host (your PC) and the VM can talk.	No	Transferring files safely between Host and Guest.
Internal	VMs can only talk to each other; the Host is blocked.	No	Isolated labs (Malware, Domain Controllers).

Malware Lab Setup

Hardware Setup Requirement

- System S\Hardware
- 64 bit Intel or AMD processor
- Virtualization enabled Bios
- 8 GB (Minimum RAM)
- 40 GB (Free Disk Space)
- Guest OS
 - Windows 11
- Hypervisor
 - Vmware
 - Virtual box

Virtualization Support in Hardware

- **Virtualization Support** is a hardware feature that allows your physical processor (CPU) to act as multiple independent "virtual" units.
- On Linux
 - Open your terminal and run: `lscpu | grep Virtualization`
 - If you see **VT-x (Intel)** or **AMD-V / SVM (AMD)**, the hardware supports it.
- On Windows
 - Press Ctrl + Shift + Esc to open Task Manager.
 - Click the Performance tab and select CPU.
 - Look for Virtualization at the bottom right. It should be enabled.
 - If Disabled: You need to enter the BIOS to turn it on.

Virtualization Support in Hardware

- The BIOS (or the modern **UEFI**) is the "firmware" that connects your computer's physical hardware to your software.
- UEFI (Unified Extensible Firmware Interface) .
- UEFI is the first software that runs when you power on your computer.
- Virtualization is a feature built into your CPU (the processor), but by default, it is often locked for security reasons. The BIOS is the only place where you can give your operating system permission to "unlock" those special hardware instructions.

Virtualization Support in Hardware

- **Intel VT-x** and **AMD-V** are "Hardware-Assisted Virtualization" technologies. They are essentially specialized instruction sets built directly into your CPU to make running virtual machines (VMs) faster, more secure, and more stable.

Virtualization Support in Hardware

- In the x86 architecture, **Protection Rings** are a mechanism built into the CPU to prevent software from accidentally (or maliciously) destroying the system. They act like layers of security clearance
- **Ring 3 (User Mode):** The "Lobby." This is where your apps (Chrome, Discord, Games) live. They have no direct access to the hardware. If an app crashes here, it doesn't kill the computer
- **Rings 1 & 2:** Historically used for device drivers, but modern OSs like Windows and Linux don't use them. They mostly sit empty today.
- **Ring 0 (Kernel Mode):** Ring 0 is the "Control Room." It is the most privileged level within a standard Operating System. It has absolute power over the CPU and memory. It can talk directly to your hard drive, network card, and monitor.

Virtualization Support in Hardware

- Normally, an Operating System (OS) like Windows wants "Ring 0" access the highest level of privilege to talk to your hardware.
 - The Conflict: When you run a VM, you have two OSs (Host and Guest) fighting for Ring 0.
 - The Solution: These technologies create a new, even higher level of privilege (sometimes called "Ring -1"). The Hypervisor (VirtualBox/VMware) sits in this new spot, directing hardware requests from both OSs so they don't crash into each other.

Virtualization Support in Hardware

- **Ring -1(Hypervisor Mode)**
- In virtualization the "Guest" OS (the one inside the VM) thinks it is the owner and tries to run in Ring 0. But the "Host" OS is already using Ring 0.
- To solve this, Intel and AMD created a "hidden" layer deeper than Ring 0, colloquially called Ring -1 (technically called VMX Root Operation for Intel).
- The Hypervisor allows multiple Guest OSs to think they are in Ring 0. When a Guest OS tries to do something privileged (like rebooting the "hardware"), the Hypervisor in Ring -1 intercepts that request and "fakes" it so the physical computer stays stable.

This layer is what keeps your malware analysis lab "contained." The malware can destroy the virtual Ring 0, but it cannot see or touch the Ring -1 layer that controls the physical machine.

Virtualization Support in Hardware

Level	Name	Common Role	Scope of Impact
Ring 3	User Mode	Apps (Chrome, Word)	Only the app crashes.
Ring 0	Kernel Mode	OS Kernel (Windows/Linux)	The whole OS crashes (BSOD).
Ring -1	Hypervisor	VMware / VirtualBox / KVM ¹³	Can manage/reset multiple OSs.

Network Adapter

- In a malware analysis lab, the Network Adapter is the most important "gatekeeper." It determines whether the malware can talk to your host machine, your home Wi-Fi, the internet, or other virtual machines.
- When you configure a VM in VMware or VirtualBox, you are presented with several modes. For malware analysis, the choice is usually between **Host-Only** and **Internal**, while **NAT** and **Bridged** are avoided.

Bridge Adapter

- In Bridged mode, the VM acts as a separate, physical device on your real network.
- The VM gets its own IP address directly from your home router (e.g., 192.168.1.15).
- The malware is "loose" on your actual network. It can scan your phone, your family's laptops, and your smart home devices.
- Use Case: Never used for malware. Only used if you want a VM to act as a real server on your network.

NAT - Network Address Translation

- The VM "hides" behind your host computer's IP address.
- It is like your host machine is a private router for the VM. The VM can reach the internet, but the internet (and your local network) cannot easily reach into the VM.
- While the malware can't easily see other devices on your Wi-Fi, it can reach the public internet. It might join a botnet or download malicious payloads.
- Use Case: Temporarily enabling it to update tools or download a sample, then turning it off immediately.

Host-Only Adapter

- This creates a private, "invisible" network between the VM and your physical host machine.
- The VM and the Host share a virtual switch. They can talk to each other, but the VM has zero access to your Wi-Fi or the internet.
- It is safe from the internet, but if you don't configure your host's firewall properly, the malware could theoretically try to attack your host machine through this shared connection.
- Use Case: Standard lab setup where you need to move files between your host and the VM.

Internal Network

- This is a "bubble" that only exists between virtual machines.
- How it works: The virtual machines can see each other, but they cannot see the host machine, and they cannot see the internet. It is a closed loop.
- **This is the professional standard for malware analysis.**
- The "Dual-VM" Setup: Usually, analysts put a Windows Victim and a REMnux Gateway on the same Internal Network. The Windows machine tries to go online, but all its traffic is caught and "faked" by the REMnux machine.

Summary of Network Adapters

Adapter Mode	Internet Access	Host Access	LAN Access	Lab Safety
Bridged	Yes	Yes	Yes	Unsafe
NAT	Yes	No (mostly)	No	Risky
Host-Only	No	Yes	No	Safe
Internal	No	No	No	Safest

Static Analysis

Static Analysis

- Hashing Analysis
- Strings
- Obfuscated Strings
- Library and functions
- Malware Signatures

Hashing

- Hashing is the process of converting input data of any size (such as a file, message, or malware sample) into a fixed-length alphanumeric value known as a **hash value** or **message digest**, using a mathematical hash function. Even a small change in the input results in a completely different hash value.
- **Common Hash Algorithms**
 - **MD5 (128-bit)**: Fast but vulnerable to collisions; not recommended for security, still used for identification.
 - **SHA-1 (160-bit)**: Weaker, deprecated for secure applications.
 - **SHA-256 / SHA-512**: Strong, widely used in forensics and security.
 - **SHA-3**: Newer standard with improved security design.

Hashing in Malware

- In malware analysis, hashing is primarily used for identification and classification of malware samples.
 - Uniquely identify malware samples
 - Detect known malware by comparing hashes with threat intelligence databases
 - Track variants of the same malware family
 - Maintain integrity of malware evidence
- Antivirus engines and security tools use hash values as malware signatures. If a file's hash matches a known malicious hash in a database, the file is flagged as malware.
- **Limitation:**
Even a minor modification (packing, obfuscation) changes the hash, allowing malware to evade simple hash-based detection.

Hashing in Digital Forensics

- Evidence integrity verification
- Chain of custody maintenance
- Court admissibility of digital evidence

Strings

- String Analysis is the process of extracting readable characters and words from the malware.
- It gives information about malware functionality.
- Strings are in ASCII and Unicode format(Specify the format you want to extract)
- Important strings include
 - File namesURLs
 - IP addresses
 - Registry keys
 - API function names
 - Error messages

Why does .exe contain strings?

- A .exe file is a Windows Portable Executable (PE) file.
- It is not just machine code.
- It is a structured container that holds many different things.

Section	Contains
.text	Machine code (CPU instructions)
.rdata	Read-only data (strings, constants)
.data	Global variables
.rsrc	Resources (icons, dialogs, version info)
.idata	Import table (API names)

- Here the API names, dialogs, version could be strings, they are used to execute the strings directly by COU but aid CPU in executing instructions e.g function call.

Life cycle of .exe file execution

- Developer writes source code (.c, .cpp)
 - Human-readable, not executable.
- Compiler translates source code → machine code
 - Machine code fragments
 - Unresolved external references
 - Data & strings
- Linker combines .obj files + libraries
- Builds **Portable Executable (PE)** format

Linker and Disassembler

- The linker stitches together compiled code and libraries into a complete executable. The linker:
 - Combines object files
 - Adds Windows libraries
 - Builds the **PE structure**
- A **disassembler** does the **reverse of compilation**.
 - 55 8B EC in machine code is equivalent to `push ebp, mov ebp, esp` assembly code

Types of strings

- Malware strings can broadly be classified into:
 - **ASCII Strings:** Plain readable text (e.g., URLs, file paths)
 - **Unicode Strings:** Wide-character strings commonly used in Windows malware
 - **Encoded / Obfuscated Strings:** Hidden or encrypted strings to evade detection
- ASCII → always 1 byte per character
- UTF-16 → usually 2 bytes per character (Windows default)
- UTF-8 → variable (1–4 bytes)
- UTF-32 → always 4 bytes

String Commands

- String Command
 - `> strings.exe sample.exe`
- Unicode Strings:
 - `> strings.exe -n 5 -u sample.exe`
- Minimum length (reduce noise):
 - `>strings.exe -n 5 -u sample.exe`
- Save output to other file:
 - `>strings.exe -n 5 -u sample.exe > strings_output.txt`
- Find URLs / C2:
 - `>strings.exe -u sample.exe | findstr /i "http https" (for APIs CreateFileW)`
- Registry Artifacts:
 - `>strings.exe -u sample.exe | findstr /i "software\\microsoft"`
- Suspicious Commands:
 - `>strings.exe -u sample.exe | findstr /i "cmd powershell"`

Obfuscated Strings

- Line in program : `char s[] = { 0x1D, 0x10, 0x19, 0x19, 0x1A };`
- Decode Code:

```
for (int i = 0; i < 5; i++) {  
    s[i] ^= 0x55;  
}
```

After decoding (runtime) : HELLO

```
char s[] = {0x36, 0x38, 0x31}; // stored, obfuscated

void run_command() {
    for (int i = 0; i < 3; i++) {
        s[i] ^= 0x55; // decode code runs now
    }
    printf("%s\n", s); // prints "cmd"
}

int main() {
    run_command(); // decode happens here
    return 0;
}
```

